

APP2000 - Arbeidskrav 1

GRUPPE 12

Brage Langkås - 216558

Tor Alvar Andersen - 284871

Cecilie Halvorsen - 286406

Daniel Larsen - 284882

Oppgave 5 - L1

Vi valgte å teste Github Copilot via Agent vinduet til VS Code, og Gemini via Agent vinduet til Antigravity.

Begge ble gitt samme prompt:

```
Bruk KI for å generere en prototype for landingssiden til [prosjektoppgaven i  
APP2000 for studieåret 2025/2026  
(turapp)](https://dbsys.info/2000/prosjekt/oppgave2526.html). Sørg for at  
prototypen bygges kun med HTML og CSS, altså en statisk nettside uten  
backend/database
```

Sørg for at du går inn på lenken og analyserer nettsiden, ikke bare anta hvordan
den ser ut.

PDF-fil med resultatene nettsidene ligger nedenfor, Copilot har gått for en mye enklere løsning, med lite detaljer, ingen bilder, og nokså steril layout.

Dette stemmer overens med at Copilot er låst på Auto, som valgte å gå for GPT-5-mini til denne oppgaven. Som er en simplere modell.

Gemini har gått for mye mer innhold, generert bilder der det trengs, og prøvd seg på mer ambisiøse moduler. Som det søkbare kartet og fellestur oversikten med værddata.

Begge modellene hadde nok laget et betydelig bedre resultat med relevante SKILL.md filer og et bestemt design-system, samt flere iterasjoner.

A) Effektivisere utvikling av webapplikasjoner

B) Utvikle webapplikasjoner med "KI-funksjonalitet"

- **Prompt** - Beskriver det bruker skriver til KI-modellen. Relevant til både A og B.
- **Token** - Alt en KI-modell gjør blir delt opp i tokens. Prompts, "tenking", oppgaver, alt brytes ned til tokens og så beregner modellen den mest sannsynlige neste token, og setter disse sammen. Relevant til A og B
- **Copilot** - En KI-modell (Microsoft Copilot) eller Github Copilot som er en samling av modeller (GPT og Claude) fra OpenAI og Anthropic. Relevant til A og B.
- **Coding Agent** - Agents er modeller som er mer hands-on enn en tradisjonell KI, de er gjerne integrert i IDE'en slik at de kan få tilgang til prosjektmapper for å kunne opprette og endre innhold direkte. Relevant til A og B

- **Agent Skill** - Markdown-filer andre personer/bedrifter har utviklet for at agenten skal få bedre ytelse i spesialiserte oppgaver. Kan hentes via en packet manager som npm, og lagres som en SKILL.md fil, CLAUDE.md om man bruker Claude. Relevant til A og B
- **AI API** - API-nøkkel som brukes til å kommunisere med KI-modeller. Disse kan integreres i en webapplikasjon for å lage en chatbot f.eks. Relevant til B
- **AI SDK** - SDK står for Software Development Kit, dette er et bibliotek som håndterer API-kallene for en KI-funksjon. Relevant for B
- **AI CLI** - En agent som opererer direkte i terminalvinduet. CLI står for Command Line Interface, og er en alternativ måte å samhandle med en KI-modell. Noen foretrekker dette over å samhandle med agenten via et grafisk grensesnitt som i VSCode. Relevant for A og B
- **Model Context Protocol (MCP)** - En server tilkobling til agenten som gir den utvidede funksjoner, nå kan verktøy som Figma, Github, og Playwright opprette 1 MCP standard som fungerer med alle KI-verktøy som støtter MCP istedenfor unike integrasjoner til hvert KI-verktøy. Relevant til A og B.

Kodeassistentene hjelper mest under utviklingen. Prompts, tokens, API-er, SDK-er, skills og MCP'er kan brukes både i utviklingsverktøy og i apper med KI-funksjoner

Oppgave 6 - L1

Vibe coding

Etter å ha spurt Claude om forskjellen mellom Vibe coding og SDD, fikk jeg som svar at Vibe coding er mer at man kommuniserer med modellen gjennom naturlig språkbruk. Man lar modellen generere koden, ser om den fungerer og fortsetter man med prompts til man er fornøyd. Man leser nødvendigvis ikke gjennom kodelinjene, men man forholder seg kun til resultatet.

SDD

Spec driven development snur om på rekkefølgen. Vi gir modellen først en spesifikasjon om hva som skal bygges, hvilke krav og kriterier som gjelder og hvordan det skal henge sammen. Vi bruker spesifikasjonen som utgangspunkt for koden vi som skal skrives. Spesifikasjonen ligger i repoet, versjonshåndteres og oppdateres. Det er her man utfører endringer, koden blir mer som et produkt av spesifikasjonene og ikke omvendt.

Microsoftbloggen gir oss klare steg for SDD-utvikling:

Constitution → prinsipper, standarder og overordnede rammer
Specify → krav, brukssituasjoner og akseptansekriterier
Clarify → rydde opp i uklarheter, avhengigheter og spesialtilfeller
Plan → planlegge arkitektur, flyt og tekniske begrensninger
Tasks → dele planen opp i konkrete utviklingsoppgaver
Implement → bruke KI til å generere/forbedre kode og tester
Validate → kontrollere resultatet mot spesifikasjonen.

Dette understrekes med *“Spec quality = output quality”*

Tatt i bruk Github Copilot via Agent Manageren i VS Code for denne oppgaven, modellen ble gitt denne prompten:

```
https://dbsys.info/2000/prosjekt/oppgave2526.html

Følg lenken for oppgavebeskrivelse, så skal du opprette en liste med
SDD-spesifikasjoner (Spec-driven development) for denne oppgaven etter
reglene gitt i disse to linkene:
https://developer.microsoft.com/blog/spec-driven-development-ai-native-engineering#github-spec-kit
https://www.deeplearning.ai/courses/spec-driven-development-with-coding-agents

Analyser begge kildene for hvordan SDD'en skal formidles.
```

Den gir en nokså komplett SDD-sheet allerede på første prompt, dette er nok fordi det er en god oppgavebeskrivelse samt regler for utforming via lenkene.

Fremdeles er det nok lurt å iterere videre på listen sammen med agenten, den har stilt noen spørsmål i bunnen som må oppklares før [plan.md](#) ferdigstilles f.eks.

Oppgave 8 - L1

Nettsiden kan nås her: <https://lngks.github.io/>

Denne versjonen er laget på standard html og css.

Først har det blitt lagt en `<div class="grid"></div>` rundt `<header>`, `<sidebar>`, `<main>` og `<footer>` slik at alle ligger inne i denne containeren.

Da kan vi sette opp CSS'en slik

```
.grid {
```

```

display: grid;
min-height: 100vh;
grid-template-rows: auto 1fr auto;
grid-template-columns: 260px 1fr;
grid-template-areas:
    "header header" /* Her kan oppsettet endres*/
    "sidebar main"
    "footer footer";
gap: 1rem;
}

.header { grid-area: header; } /* Her assigner vi tags til grid-area */
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }

@media (max-width: 768px) { /* Dynamisk oppsett til mindre skjermer */
    .grid {
        grid-template-columns: 1fr;
        grid-template-areas:
            "header"
            "sidebar"
            "main"
            "footer";
    }
}

```

Oppgave 1 - L2

Input-felt og en Oppdater-knapp i HTML først, også vil JavaScript håndtere resten. `updateContent()` vil hente tekstinnehold fra feltet, og opprette et p-element med tilsvarende innhold.

`buttonListener()` fester en listener på både `inputField` og `button`, slik at vi kan kjøre `updateContent()` enten ved at bruker trykker på knappen, eller trykker "Enter" på tastaturet mens `inputField` er i fokus.

`paragraphListener()` ser etter trykk på p-elementene som er opprettet og vil kjøre `removeContent()` om bruker trykker på et p-element.

Oppgave 4 - L2

Det er ikke tilstrekkelig med kun klient-side validering, siden brukeren kan endre eller omgå JavaScript og HTML i nettleseren. Burde man også implementere server-side validering for å sikre at dataen er gyldig før den blir behandlet av serveren.

Oppgave 5 - L3

Fordeler med TypeScript

- Man definerer typene (f.eks. Coordinates, Place og Cabin) på forhånd, noe som gjør at feil oppdages tidligere.
- Kompileringen hjelper til med å fange feil tidlig.
- Bedre feilmeldinger i VS Code gjør det lettere å identifisere problemer.
- Funksjoner dokumenterer eksplisitt hvilken type data de forventer.

Ulemper med TypeScript

- Krever kompilering mellom hver endring (selv om enkelte rammeverk automatiserer dette).
- Oppsettet er mer omfattende enn vanlig JavaScript (krever installasjon og konfigurasjon som tsconfig.json).
- Typene kontrolleres kun ved kompilering, ikke under kjøring. TypeScript kan ikke garantere at eksterne data (f.eks. fra data.json) er riktig formatert uten et valideringsbibliotek.

Testing

- Verifiser at data hentes og vises korrekt i localhost ved å klikke på knappen.
- Test at X-knappen lukker kortet.
- Bruk DevTools i nettleseren for å sjekke konsollet for JavaScript-feil.